
MP2: Vertex Shader and Object Space

Due Time: Refer to the course web-site

Objectives

Verify that we understand and are able to work with:

- Unity specific skills:
 - Existing and new components (Mono-behavior scripts) to refine and accomplish specific application functionality
 - Interactively passing parameters from CPU to shaders in the GPU
 - Material instancing when changed by the associated game object
 - Appreciate and use pre-define shader variables in Unity to create simple animation
- Shader and relevant general concepts:
 - Bit-masks and linear blending
 - Parameters for UV file texture mapping
 - Simple multi-pass shaders
 - Object space and algorithmically modify object geometry
- **Concrete understanding of object space is key!**

Approach

You will work with two given unity components:

- MyMesh: where there are many vertices to work with and you can change the mesh resolution
- CameraManipulation: where you can interactively control the camera (for runtime examination)

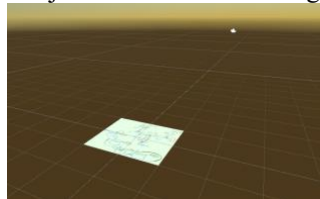
You will create a vertex shader to process MyMesh geometry and generate a pseudo wave-like object. The CameraManipulation script allows interactive camera manipulation in examining your results. The following are the details.

MyMesh is a NxN mesh (controlled by the New Resolution, with default of 20x20):

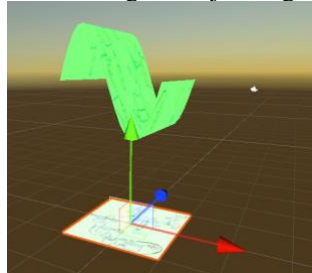
- **Object Space:** On the X/Z plane, located at the origin, each side is of size 2. This says, the object is defined in the range of (-1, 0, -1), and (1, 0, 1).

Your Shader:

- **Pass 1:** render the usual mesh object in the world, e.g., as shown in the following screenshot, where, the object to world transform is a simple scaling of 10x, so that the object is of size 20 (2 x 10). In the world space, the object is defined in the range of (-10, 0, -10) to (10, 0, 10).



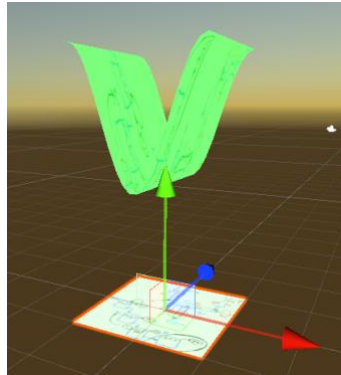
- **Pass 2:** render an algorithmically computed sinusoidal geometry at **Y_Offset** (offset measured in object space) above the original object, e.g., as shown by the greenish object in the following screenshot:



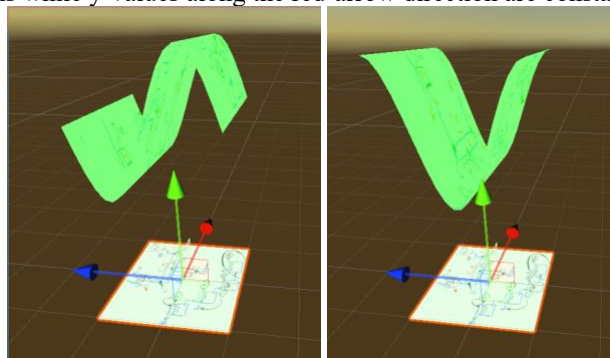
Your Vertex Shader:

- Must support creating a sinusoidal geometry with the following user control parameters:
 - **Type:** A sine or a cosine curved object
 - **Period:** number of periods to show with the given object space range of -1 to 1
 - **Amplitude:** Amplitude of the curved object

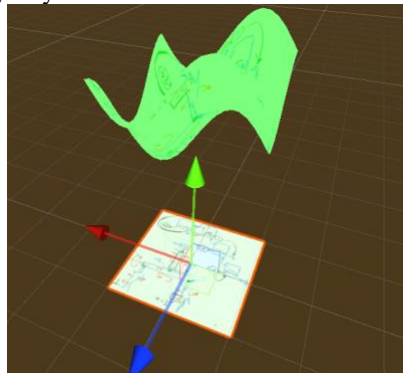
For example, the above green-ish object is created as a sine-object, with period of 1 (shows one period in the object space range), and object space amplitude of 1.0. The world coordinate size of the object is scaled by the GameObject transform to 10. The following screen shot shows the results of **Type** being a cosine function:



- **Support in both X and Z directions:** Notice that in the previous screenshots, the sinusoidal function is defined along the x-axis: the y-value changes are along the red-arrow direction while the blue arrow direction has constant y-values. You must support same functionality along the z-axis direction, as shown in the following screen shots for the sine and cosine functions: y-value changes are along the blue-arrow directions while y-values along the red-arrow direction are constant.



- **Support the blending of X and Z directions:** You must define a **XZBlend** to support the blending of the above sinusoidal functions. A XZBlend of 1 means showing only the X-direction function, and 0 means showing only the Z-direction function. The following screenshot is a XZBlend value of 0.5:



- Animating the geometry: Your shader must support the animation of the sinusoidal shape by using the Unity [build-in shader parameter](#): `_Time`. Please use `_Time.x` to modulate the angle you use for computing angles for the sinusoidal functions.

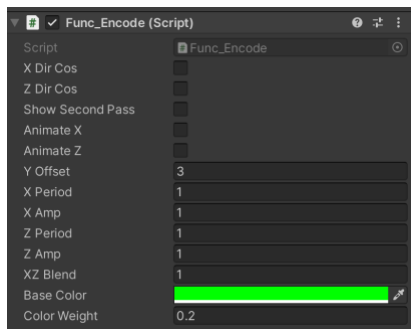
Your Fragment Shader: lastly, let's begin working with color computation. As a straightforward approach, your shader must accept a main texture, and support the following two parameters for computing a final color in the fragment shader:

- **BaseColor:** this is a color
- **ColorWeight:** this represents how to blend BaseColor with the sampled texture color. A ColorWeight of 1 means you should show the texture color only, while a ColorWeight of 0 means you should only show the BaseColor.

Shader function flag support: To conserve the number of parameters passed to the shader, you must define one unsigned-integer with the following bit-mask:

- Bit 0: 0 means X-direction should be a sine function, while a 1 means cosine function
- Bit 1: 0 means Z-direction should be a sine function, while a 1 means cosine function
- Bit 2: not used
- Bit 3: do not (0) or render (1) the modified geometry that is computed by the second pass
- Bit 4: do not (0) or animate (1) in the X-direction
- Bit 5: do not (0) or animate (1) in the Z-direction

Script to encode the flag and set the material: It is non-trivial to encode the above bit-mask flag. You will implement a script for the MyMesh object to support setting of all of the above parameters during runtime. The following is a screenshot of the public variables from my implementation:



Reference Results

- Here is [a video \(MP2-Part1\)](#) showing:
 - 0 to 0:15: Showing Second Pass and Y-offset of second pass
 - 0:15 to 0:38: X-direction Sine/Cosine Period and Amplitude modifications
 - 0:38 to the end: Z-direction Sine/Cosine Period and Amplitude modifications
- Here is [a video \(MP2-Part2\)](#) showing:
 - 0 to 0:30: Showing animating in X- and Z- directions and blending of the two
 - 0:30 to the end: Changing of color and color weight

Hints

This assignment is whole lot more straightforward than the specification may be suggesting. The number of lines of code is actually *very* small, but maybe a little tricky. Here are some hints:

- The sinusoidal shape is created by modifying the vertex y-positions. Since the object space is between (-1, 0, -1) to (1, 0, 1), we know that the x and z vertex values are between -1 to +1. For X-direction shape, corresponding y-values are computed based on an expression *similar* to:

$$\text{vertex.y} = \sin(\text{vertex.x} * 2 * \text{PI});$$

You can try the above and observe a corresponding sinusoidal shape. For a z-direction shape, you will simply replace the x by z.

- *Blending* of x and z directions simply means a linear combination (based on XZBlend) of the separate y values computed.
- Animating the shape can be accomplished by changing the theta for the sinusoidal function continuously. E.g.,

$$\text{vertex.y} = \sin((\text{vertex.x} - _Time.x) * 2 * \text{PI});$$

Credits Distribution

	Credits
1 Settings in Unity	
Support MyMesh GameObject	5
Proper material setting script	10
Updates all material parameters	
2 Bitwise Flags	
Proper bitwise flag supported	10
3 Vertex shader sinusoidal shape	
Type, Period, Amplitude	15
Support Animation and on/off	10
Support creation in X and Z directions	10
4 Blending Support	
Support X/Z shape blending	10
Color blending in fragment shader	10
5 Multi-pass shader	
Shape creation is in second pass	5
Second pass can be switched off	5
6 Submission	
Proper project name	5
Four folders in submission	5
Total	100

This programming assignment counts 7% towards your final grade for this class.